
```
def myFunctionOrMethod(oLocalSquareParam):  
    # oLocalSquareParam now refers to wherever the argument refers to  
    <body of myFunctionOrMethod>  
  
myFunctionOrMethod(oSquare1) # call the function and pass in the object
```

3. When an object is put into a container such as a list or a dictionary:

```
myList = [oSquare1, someValue, someOtherValue]
```

If `oSquare1` already refers to an object, after executing this line, the list contains an additional reference to the same `square` object.

Decrementing the Reference Count

Decrementing the reference count also happens in a few ways. To demonstrate this, let's create an object and increment its reference count:

```
oSquare1 = Square(20, BLACK)  
oSquare2 = oSquare1  
myList = [oSquare1]  
myFunctionOrMethod(oSquare1) # call the function and pass in the object
```

When `myFunctionOrMethod()` starts, the reference to the object is copied into a local parameter variable for use inside the function. The reference count of this `square` object is currently 4: two object variables, one copy inside a list, plus a local parameter variable inside the function. This reference count can be decremented:

1. When any variable that refers to an object is reassigned. For example:

```
oSquare2 = 5
```

2. Whenever a local variable that refers to an object goes out of scope. When a variable is created inside of a function or method, the scope of that variable is limited to that function or method. When the execution of the current function or method ends, that variable literally goes away. In this example, when `myFunctionOrMethod()` ends, the local variable that refers to the object is eliminated.